

Pipeline di Parsing Web e Valutazione con LLM-as-Judge

Denis Fava
2018687

Riccardo Pucci
1994172

Alessandro Cantaressi
2129091

1 Obiettivo del progetto

Data una URL di uno dei quattro domini supportati (`en.wikipedia.org`, `www.basketball-reference.com`, `global.morningstar.com`, `it.tradingview.com`), il sistema scarica la pagina, ne estrae il contenuto informativo in Markdown e ne misura la qualità in due modi indipendenti: con metriche automatiche rispetto a un Gold Standard costruito a mano e con il giudizio di un LLM locale. Tutto è esposto via API REST e Web UI, in quattro container avviabili con un comando.

La scelta dei domini non è neutra: un'enciclopedia di prosa, un archivio statistico di tabelle, un sito editoriale finanziario e una *single page application* pongono problemi di parsing diversi, e come si vede nella sezione 4 mettono in crisi anche le metriche.

2 Organizzazione del codice

Il sistema è orchestrato da `docker-compose.yaml` in quattro servizi: **backend** (8003), **frontend** (8004), **mariadb** (3306) e **ollama** (11434). Dopo `docker compose up --build` non serve nessun passaggio manuale: lo schema nasce con il container MariaDB, Ollama scarica il modello dal proprio endpoint e il backend popola il database dai JSON di `gs_data/`.

I parser sono a oggetti. `BaseParser` definisce `parse()` come *template method* — pulizia dell'HTML, estrazione, conversione in Markdown e normalizzazione fissate una volta sola — e lascia astratto il solo `extract_blocks`, l'unica cosa che cambia fra domini. Il dispatch sta in `registry.py`: in `server.py` non c'è nessun `if domain ==`, e aggiungere un dominio significa una sottoclasse e una riga di registro.

La logica sta in `backend/src/services/`,

Dominio	token F1	seq. F1	judge
<code>morningstar.com</code>	0.965	0.951	5.00
<code>basketball-reference.com</code>	0.949	0.922	3.20
<code>wikipedia.org</code>	0.930	0.951	4.80
<code>tradingview.com</code>	0.889	0.890	2.80

Tabella 1: Metriche medie per dominio, dieci pagine ciascuno.

una responsabilità per modulo: `database.py` (connessione e pool), `repository.py` (tutte le query, e solo lì), `evaluator.py`, `judge.py`, `bootstrap.py`, `text_utils.py` e `markdown_utils.py`. `server.py` resta un livello sottile: espone i quattordici endpoint, ognuno con il proprio `response_model` Pydantic, e traduce gli errori in codici HTTP. Il frontend, in Jinja2, parla solo con le API REST e non conosce il database.

3 Valutazione dei parser

Il Gold Standard contiene **40 entry**, dieci per dominio, ciascuna con URL, dominio, titolo, HTML grezzo e testo di riferimento copiato a mano. La valutazione lavora sempre sull'HTML statico nel database: lo chiede la consegna ed è l'unico modo di avere numeri confrontabili su siti che cambiano ogni giorno.

Le metriche sono due. `token_level_eval`, obbligatoria, confronta gli *insiemi* di token; `sequence_eval`, aggiunta da noi, lavora sulle *sequenze* calcolando precision, recall e F1 sulla più lunga sottosequenza comune con `difflib.SequenceMatcher`. Esiste perché la prima ha un punto cieco: sugli insiemi, ordine e ripetizioni non contano, e un testo con un paragrafo ripetuto tre volte ottiene token F1 1.000 e sequence F1 0.737.

Tutti e quattro i domini superano la soglia di 0.80 che la consegna classifica come “Buono”. Il più difficile è TradingView: le schede sono grigie

Modello	media	JSON	s	corr.
ministral-3:3b	4.17	100%	13.1	-0.926
llama3.2:3b	4.00	100%	7.6	-0.676
phi4-mini	3.33	100%	18.6	-0.621
qwen3:4b	1.00	0%	27.8	—
gemma4:e2b	1.00	0%	30.7	—

Tabella 2: I cinque modelli ammessi. “corr.” è la correlazione di rango fra judge e F1; l’1.00 dei modelli senza JSON valido è il fallback, non un giudizio.

generate da React, con classi CSS che contengono un hash di build e cambiano a ogni rilascio. Il parser si aggancia perciò agli attributi `data-qa-id`, che il sito usa per i propri test e restano stabili, con una whitelist dei widget informativi invece di una blacklist del rumore.

Dalla costruzione del Gold Standard sono emerse le due osservazioni più istruttive del lavoro. La prima è la **coerenza del criterio di copiatura**: su Basketball-Reference quattro pagine su dieci erano state trascritte includendo le tabelle statistiche, che il parser esclude di proposito, e omettendo la coda della pagina, che il parser include. L’F1 medio era 0.678, con minimi a 0.096: non un difetto del codice, ma una divergenza fra chi scriveva il riferimento e chi scriveva il parser. Uniformato il criterio, il dominio è passato a 0.949 *senza toccare una riga di parser*. La seconda sono i **caratteri invisibili**: il testo copiato dal browser trascina marcatori Unicode di direzionalità e byte order mark, che TradingView mette attorno a ogni numero, e poiché la metrica confronta token separati da spazio lo stesso numero risultava diverso da sé stesso. La normalizzazione sta in `text_utils`, nell’unico punto in cui un `gold_text` entra nel database e non dentro la metrica: così nel database esiste una sola versione del testo.

4 Valutazione dei judge

Il judge dialoga con Ollama su `/api/chat` con `"stream": false`. I due messaggi hanno ruoli distinti: il `system` porta ruolo, criteri di penalizzazione e scala, lo `user` solo i due testi. Il formato è imposto con `"format": "json"`, la temperatura è bassa perché serve un giudizio ripetibile, e i testi sono troncati a 2000 caratteri: la consegna lo consente e su CPU è la differenza fra una valutazione che termina e una che va in `time-out`. Se il JSON non è leggibile scatta il fallback richiesto: punteggio minimo e `parse_ok` a falso.

Due modelli su cinque non hanno prodotto una

singola risposta valida, ed erano anche i più lenti: generano token di ragionamento e il limite di generazione li troncava prima del JSON — “non funziona” e “è configurato male” si assomigliano molto dall’esterno. Abbiamo scelto `ministral-3:3b`: media più alta, piena affidabilità sul formato e correlazione più netta con la metrica, accettando di essere più lenti di `llama3.2:3b`.

La correlazione merita attenzione: il segno è *negativo* per tutti e tre i modelli affidabili e per il migliore vale -0.926, cioè il judge ordina le pagine quasi al contrario dell’F1. Lo stesso disaccordo si legge nella tabella 1, dove Basketball-Reference ha il secondo miglior F1 ma il secondo peggior judge. Le due valutazioni rispondono a domande diverse: la metrica misura *quanto* è stato estratto, il judge se il risultato *somiglia a un testo informativo*. Su pagine che sono griglie di dati la seconda ha risposta bassa anche a estrazione corretta — non a caso i domini con judge più basso sono i due fatti di tabelle. È il motivo per cui la consegna chiede entrambe: un judge è un secondo parere, non un sostituto.

5 Organizzazione del database

Il database è MariaDB, interrogato con MariaDB Connector/Python. Tutte le query stanno in `repository.py` e usano esclusivamente segnato ? con tupla di valori: nessuna query è costruita per interpolazione di stringa, che è la porta d’ingresso classica della SQL injection.

Lo schema, in `mariadb_data/init.sql`, ha cinque tabelle. `web_resources` e `gold_standard` hanno nome e colonne fissati dalla consegna e sono legate uno-a-uno: la colonna `url` è insieme chiave primaria ed esterna, con `ON DELETE CASCADE` a implementare il requisito per cui `DELETE /web_resource` rimuove anche il gold standard. Le altre tre — `parsed_documents`, `evaluations`, `judgements` — sono nostre e servono a `/db_stats`, che per specifica legge dati già calcolati.

Un dettaglio non ovvio: InnoDB limita una chiave a 3072 byte e in `utf8mb4` un carattere ne occupa fino a quattro, quindi una `VARCHAR(2048)` come chiave primaria supererebbe il limite. Poiché gli URL sono ASCII per definizione (RFC 3986), la colonna è `CHARACTER SET ascii COLLATE ascii_bin: 2048 byte`, lunghezza da specifica invariata e confronto case-sensitive.

6 Bonus/Extra

Interfaccia web. La UI è un servizio a sé, in FastAPI e Jinja2, e non conosce né database né parser: tutto passa dalle API REST. Le pagine sono quattro, una responsabilità ciascuna, e lo stato dei tre componenti resta visibile da qualunque pagina in una barra laterale. Quella barra costerebbe due chiamate REST per richiesta, quindi `/status` e `/domains` sono raccolte in un unico punto con cache di dieci secondi e timeout corto: i domini cambiano solo al riavvio del backend, e una barra laterale non deve far aspettare la pagina.

Errori mostrati all'utente. Nessuna eccezione esce dal livello che chiama il backend: ogni chiamata restituisce la coppia (dati, errore), e l'errore porta due campi distinti, il messaggio in italiano che va in pagina e il dettaglio tecnico, che resta in un riquadro richiudibile. Errori con la stessa causa vengono fusi. Con il backend fermo il sistema resta navigabile: le pagine rispondono, gli indicatori diventano rossi e l'utente legge cosa non funziona.

Il costruttore del Gold Standard. L'HTML viene salvato al momento del download e non viaggia nel form: su una pagina da mezzo milione di caratteri, rispedirlo come campo nascosto supera il limite di una parte multipart e il salvataggio fallisce. Accanto al riquadro in cui si incolla il riferimento mostriamo l'output del parser dichiarando che serve a scegliere *quali sezioni* copiare, mai come sorgente: un gold standard ricavato dall'output del parser misurerebbe il parser contro se stesso.

Riproducibile e non riproducibile. I JSON di `gs_data/` sono la fonte di verità, il database una copia ripopolabile: parsing e metriche si ricalciano, un testo scritto a mano no.

Il primo avvio su database vuoto. Le 40 entry pesano 26 MB di HTML e venivano inserite in un unico `executemany`: MariaDB rifiuta i pacchetti oltre `max_allowed_packet` (16 MB) e chiude la connessione senza spiegazioni. L'inserimento è stato diviso in lotti da 4 MB lato client invece di alzare il limite lato server. Il problema si vedeva solo partendo con i volumi Docker cancellati, ed è per questo che quella prova fa parte della nostra procedura di verifica.